Winston AI platform: a comprehensive architecture audit

Winston has a solid architectural foundation — better than most early-stage AI platforms — but critical gaps in observability, model diversity, RAG pipeline ordering, and enterprise infrastructure must be addressed before it can credibly serve institutional REPE firms. The platform's domain positioning is exceptional: no competitor offers an integrated AI operating system spanning document intelligence, financial modeling, portfolio monitoring, and LP reporting for Real Estate Private Equity. However, the current technical architecture carries single-provider risk, lacks evaluation infrastructure, and misses several patterns that the best AI platforms adopted in 2025. This audit benchmarks Winston against 10 leading AI platforms, identifies 23 specific improvements, and provides a phased roadmap from quick wins to a v2 architecture that could define the category.

---

1. Architecture scorecard

Each dimension is scored 1–10 based on benchmarking against the architectures of OpenAI, Anthropic, Perplexity, Cursor, Palantir AIP, Databricks, Harvey AI, Ramp, Notion AI, and LangGraph.

Category	Score	Assessment
RAG pipeline	6/10	Parent-child chunking is well-sized but pipeline ordering is incorrect, embedding model is underpowered, and over-fetch ratio is too low. No contextual retrieval or query rewriting.
Model routing	5/10	4-lane design is well-structured with clear latency SLAs, but all lanes use GPT-4o — eliminating the primary benefit of routing (cost optimization, quality specialization). No fallback or dynamic routing.
Agent architecture	7/10	PLAN→CONFIRM→EXECUTE is clean and enterprise-appropriate. Human-in-the-loop for write operations is exactly right. Missing VERIFY step, adaptive replanning, and cross-session memory.
Tool system (MCP)	7/10	Comprehensive MCP tool coverage across 12+ categories is strong. Missing dynamic tool loading, MCP gateway governance, and security hardening. Tool count risks context window bloat.
Observability	2/10	No LLM observability platform, no prompt versioning, no RAG evaluation, no cost tracking per tenant. This is the most critical gap.
Infrastructure	5/10	Railway is adequate for current scale but lacks GPU support, has only 4 regions, and offers limited compliance controls. Supabase pgvector works for now but has no BM25 capability.
Security / compliance	3/10	No PII detection pipeline, no prompt injection defenses, no SOC 2 preparation, no cross-tenant data leakage testing. For a platform handling financial data, this is a high-priority gap.
Multi-tenancy	4/10	Likely basic tenant isolation. Missing row-level security enforcement, tenant-scoped caching, per-tenant cost attribution, and partial HNSW indexes for large tenants.
Domain intelligence	7/10	Document intelligence across 7 REPE document types is a strong differentiator. Financial modeling and MCP tools for RE-specific workflows show deep domain understanding.
Frontend / UX	6/10	Next.js 14 + TypeScript is the right stack. SSE streaming is correct. Missing structured event types for tool calls, token buffering optimization, and Vercel AI SDK integration.
Cost efficiency	3/10	No prompt caching, no semantic caching, no model tiering, no cost governance. Running GPT-4o for all query types — including simple conversation — is burning **3-5x more than necessary**.
Overall	**5.0/10**	A credible v1 architecture with the right instincts but missing the infrastructure layer that separates prototypes from production enterprise platforms.

2. What Winston gets right

Winston's architectural strengths are not trivial — several design decisions reflect genuine sophistication that benchmarked platforms like Harvey AI and Ramp share.

The 4-lane routing model with explicit latency SLAs is architecturally sound. By separating conversational (<1s), tool-backed (<4s), analytical (<8s), and deep reasoning (<20s) workloads, Winston mirrors the pattern Cursor uses with its "Auto" mode (Skywork) and Notion uses for routing across model tiers. (Notion) Most early-stage platforms route everything through a single path. Winston's lane separation provides the scaffolding for the model diversity and cost optimization it currently lacks.

The PLAN→CONFIRM→EXECUTE agent loop with mandatory confirmation for write operations is exactly what enterprise AI requires. Anthropic's December 2024 guidance explicitly recommends human-in-the-loop gates for irreversible actions. Ramp enforces the same pattern — "no money moves without human confirmation." This design builds trust with institutional users who cannot tolerate autonomous actions on financial data. It also provides the audit trail foundation that SEC examiners now expect. (New York State Bar Association)

The parent-child chunking strategy at 400/800 tokens falls within the **optimal range** identified in 2025 benchmarks. Vecta's benchmark of 7 strategies across 50 academic papers placed recursive 512-token splitting first at 69% accuracy, and Chroma research showed 88–89% recall with 400-token chunks. The existing configuration needs refinement, not replacement.

Comprehensive MCP tool coverage across RAG search, RE models, database access, documents, executions, tasks, metrics, reports, repository access, git tools, frontend actions, API calls, and environment configuration represents deep workflow integration. This breadth of tool access is comparable to Palantir AIP's ontology-driven action system, where agents read and write through governed primitives. (Palantir)

Domain-specific document intelligence across 7 REPE document types positions Winston in the highest-value AI workflow for the industry. JLL's 2025 survey reports **88% of real estate investors are piloting AI**, (JLL) and document extraction-to-model automation is the single highest-ROI application (V7 Labs) — reducing underwriting time by **up to 70%** (V7 Labs) according to multiple industry sources. (SourceForge)

---

3. Major risks that demand immediate attention

Three risks threaten Winston's production viability and must be addressed within 30 days.

Single-provider dependency on OpenAI is an existential operational risk. Winston runs exclusively on GPT-4o with no fallback provider. When OpenAI experiences outages — which occurred multiple times in 2024-2025 — Winston goes completely dark. Every benchmarked platform (Perplexity, Cursor, Harvey, Notion) uses multi-model strategies across **at minimum** two providers. Harvey routes legal drafting to models optimized for extended reasoning and research queries to models with superior recall. (Medium) Cursor's "Auto"

mode dynamically selects between GPT-4o, Claude Sonnet, and Gemini Flash. Notion routes across OpenAI, Anthropic, and Google based on task type. Winston needs an AI gateway layer (LiteLLM or Portkey) and at minimum Anthropic Claude as a fallback provider. Portkey's MCP Gateway would additionally provide centralized governance over Winston's MCP tool system.

Zero observability means Winston is flying blind in production. There is no LLM tracing, no prompt versioning, no RAG evaluation, no cost tracking, and no quality monitoring. This is not merely a gap — it is **the** defining difference between prototype and production AI. Every serious AI platform has observability: LangSmith traces every LangGraph execution, Databricks integrates MLflow for agent tracing, Databricks Palantir logs every action by human or AI agent, Palantir and Notion runs continuous LLM-as-judge evaluation across dozens of models. Without observability, Winston cannot detect quality regressions, debug failures, optimize costs, or demonstrate compliance to enterprise customers. **Langfuse** (self-hosted, open-source, SOC 2-compatible) should be deployed within days, not weeks. Maxim Articles Confident AI

Financial data without security infrastructure is a liability. Winston handles loan agreements, JV agreements, and financial models containing sensitive deal terms, investor information, and potentially PII. Yet there appear to be no prompt injection defenses (OWASP's #1 LLM risk, with **50-84% attack success rates**), Vectra AI no PII detection pipeline, no cross-tenant data leakage testing, and no SOC 2 preparation. The SEC's 2025 examination priorities explicitly list AI as a top focus area. New York State Bar Association Securities class actions targeting AI misrepresentations **doubled between 2023-2024**. New York State Bar Association Winston must implement input validation, output scanning, row-level security enforcement, and begin SOC 2 documentation immediately.

---

4. Missing systems Winston needs to build

Several foundational systems are absent from the current architecture. These are not nice-to-haves — they are table stakes for any enterprise AI platform benchmarked against the 2025 state of the art.

AI gateway layer. Winston calls OpenAI directly from the FastAPI backend. A production AI gateway provides multi-provider routing with automatic fallback, request-level observability and cost tracking, centralized rate limiting and token budget enforcement, and prompt caching at the gateway level. LiteLLM (open-source, Python-native) adds only ~8ms P95 latency while supporting 100+ providers. Portkey offers a managed option with sub-1ms latency and an MCP Gateway that would govern Winston's tool system.

Evaluation and quality infrastructure. No production RAG system should operate without continuous evaluation. Winston needs RAGAS or DeepEval integration measuring faithfulness, context precision, and answer relevancy. Deepchecks It needs a domain-specific evaluation dataset of **100+ question-answer pairs** drawn from actual REPE documents. It needs regression testing in CI/CD before any prompt or pipeline change ships to production. Databricks runs automated LLM-as-judge evaluation for every agent; Databricks Documentation Notion employs dedicated AI Data Specialists who run continuous evaluation catching regressions across models and prompts.

Agent memory system. Winston has no cross-session memory — every conversation starts from zero. For an "operating system," this is a fundamental limitation. When a user discusses Fund III's performance in one session, Winston should recall that context in the next session. Zep's temporal knowledge graph architecture (SOC 2 compliant, **18.5% accuracy improvement** over baselines on LongMemEval) is the leading enterprise solution. (arXiv) Letta (formerly MemGPT) pioneered the OS-inspired memory hierarchy where agents autonomously manage their own memory. (Letta) Either would dramatically improve Winston's continuity.

Cost governance system. With **84% of companies reporting AI costs cutting gross margins by >6%**, (Mavvrik) cost governance is not optional. Winston needs per-tenant token budgeting, cost attribution and chargeback reporting, rate limiting at the gateway level, automated model downgrade when budget thresholds are hit, and batch processing for non-interactive workloads (OpenAI's batch API offers **50% discounts**). (Maxim Articles)

Semantic caching layer. Research shows semantic caching achieves **61-69% hit rates** across query categories with >97% accuracy on hits. (arXiv) For REPE analysts who frequently ask similar questions about the same properties and funds, a Redis-based semantic cache with tenant-scoped keys could reduce API costs by **50-90%** for repetitive workloads while cutting latency from seconds to milliseconds. (Pedromebo)

---

5. How to dramatically improve RAG performance

Winston's RAG pipeline has the right components but assembles them in the wrong order, uses an underpowered embedding model, and misses several high-impact techniques proven in 2025 research. The following improvements are ordered by expected impact.

Fix the pipeline ordering immediately. The current flow — over-fetch → score threshold → hybrid search → metadata boosting → cross-encoder → MMR — applies a score threshold *before* reranking, which prematurely discards potentially relevant documents. Hybrid search should be the first stage, not a mid-pipeline step. The corrected pipeline: **(1)** Hybrid search in parallel (BM25 + vector) → **(2)** RRF fusion → **(3)** metadata boosting → **(4)** cross-encoder rerank (Cohere Rerank) → **(5)** score threshold → **(6)** MMR deduplication → **(7)** top-K selection. This reordering requires minimal code changes but prevents the system from filtering out documents that a cross-encoder might have scored highly.

Upgrade the embedding model. Text-embedding-3-small scores approximately **62.3 on MTEB** — roughly 2.3 points below text-embedding-3-large (64.6) and further behind Voyage-3-large and Cohere embed-v4 (65.2). (Ailog) For a domain where retrieval precision directly affects financial accuracy, this gap is material. Voyage-3-large leads independent retrieval benchmarks by a "wide gap." (DEV Community) Cohere embed-v4 tops MTEB with 100+ language support. (Ailog) Text-embedding-3-large offers the simplest migration path at 6.5x the cost ($0.13 vs $0.02 per million tokens) but delivers meaningfully better retrieval. **Re-embedding the corpus is required** — plan for a staged migration.

Increase the over-fetch ratio by 10x. A top-K of 5 is far too low for effective reranking. Standard guidance is to retrieve **50-200 candidates** before reranking to the final 5-20. The cross-encoder can only improve results if

it sees enough candidates to differentiate. ZeroEntropy's analysis shows reranking 75 candidates and sending the top 20 to the LLM saves ~73% on LLM input costs while significantly improving precision.

Add Anthropic's contextual retrieval. This technique prepends a chunk-specific explanatory context (50-100 tokens) generated by an LLM to each chunk before embedding. Anthropic's testing showed a **35% reduction** in retrieval failures with contextual embeddings alone, **49%** when combined with contextual BM25, and **67%** with reranking added. (anthropic) For financial documents where a chunk like "revenue grew by 3%" is meaningless without knowing which property and time period, contextual retrieval is the single highest-ROI improvement. Implementation cost is approximately $1.02 per million document tokens using prompt caching. (Anthropic)

Add true BM25 capability via ParadeDB. PostgreSQL's built-in (tsvector/tsquery) full-text search is not BM25. For financial documents filled with specific identifiers — property addresses, fund names, DSCR values, entity names — keyword-based retrieval is essential. ParadeDB brings production-ready BM25 to PostgreSQL, enabling true hybrid search without adding a separate search engine. (Paradedb) Combined with RRF fusion, this creates a three-signal retrieval system (dense vector + BM25 + metadata) that IBM research confirms is optimal. (Applied AI)

Implement query rewriting and multi-query retrieval. Generate 3-5 query variants from each user question, retrieve for each, and union the results before reranking. This dramatically improves recall for ambiguous queries with minimal implementation cost. For complex REPE questions like "how does Fund III's leverage compare to our underwriting assumptions," query decomposition into sub-queries yields far better retrieval than a single vector search.

Plan for GraphRAG in multi-hop queries. Microsoft's GraphRAG excels at questions requiring aggregation across a corpus (arXiv) — "What are the top risk factors across our portfolio?" or "Which properties have co-tenancy clauses that reference Tenant X?" For REPE, where relationships between entities (companies, properties, amounts, dates, clauses) are critical, GraphRAG provides **23% improvement in factual accuracy** over traditional RAG. (Microsoft) Start with LazyGraphRAG (Microsoft's lighter variant) before full GraphRAG implementation. This is a 6-12 month initiative, not a quick win.

---

6. Agent architecture should evolve, not be replaced

Winston's agent loop is fundamentally sound. The improvements here are additive refinements, not a redesign.

Add a VERIFY step after EXECUTE. The loop should become PLAN → CONFIRM → EXECUTE → **VERIFY**. After execution, a lightweight verification checks that the action produced expected outcomes — did the database update succeed? Does the generated model pass sanity checks? Are the extracted values within reasonable ranges? This pattern, common in Reflexion-style architectures, catches compounding errors before they propagate. For financial data, where a misread $1.5M vs $15M cascades through an entire model, verification is not optional.

Add adaptive replanning on failure. If EXECUTE fails, Winston should replan rather than simply report failure. Implement bounded retry — a maximum of 2 replanning attempts before human escalation. This moves Winston from rigid Plan-and-Execute toward a hybrid with ReAct's adaptability. (APXML) LangGraph's durable execution pattern provides a reference implementation: state is checkpointed at each step, and on failure, the system can resume from any checkpoint with an updated plan. (sparkco ai)

Implement dynamic tool loading to manage context window pressure. With 12+ MCP tool categories, injecting all tool definitions into every request bloats the context window. Research shows model performance degrades past ~15-20 tools per active context. Use semantic matching or a lightweight classifier to select only the 5-10 most relevant tools per query. Kong AI Gateway and Portkey's MCP Gateway both support semantic tool selection. Alternatively, implement a hierarchical "meta-tool" pattern where one tool dispatches to specialist tools based on the query domain.

Build a memory system for cross-session intelligence. This is what separates an "AI assistant" from an "AI operating system." Winston should maintain four memory tiers: working memory (current conversation context), episodic memory (records of past interactions and decisions), semantic memory (extracted facts about funds, properties, and investor preferences), and procedural memory (learned tool-call sequences for recurring tasks). **Zep** is the recommended solution — it outperforms MemGPT on the DMR benchmark (94.8% vs 93.4%), (arXiv) uses a temporal knowledge graph that naturally models deal timelines, and is SOC 2 compliant. (DEV Community) Start with cross-session user preferences and historical query patterns.

Reserve multi-agent coordination for Lane D only. Anthropic's research shows multi-agent systems use **15x more tokens** than single-agent interactions. For Winston's Lane A-C queries, a single well-equipped agent with good routing is more efficient. For Lane D deep reasoning — complex portfolio analyses, multi-document synthesis, IC memo generation — consider a supervisor/worker pattern where a lead agent delegates to specialized sub-agents (one for document retrieval, one for financial calculation, one for narrative synthesis) running in parallel. This is how Anthropic's Claude Research system reduced research time by up to 90%.

---

7. Product architecture gaps against the competitive landscape

Benchmarking Winston against the 10 reference platforms reveals several product-level architectural gaps that affect competitive positioning.

No ontology or semantic data layer. Palantir AIP's ontology maps raw data to real-world entities (Supply Chain Today) (properties, funds, investors, leases) with typed relationships and business logic. Databricks' Unity Catalog governs all AI assets — models, tools, vector indexes — through a centralized semantic layer. (Tredence) Winston's MCP tools interact with data directly, without an intermediate layer that encodes domain knowledge. For REPE, an ontology mapping properties → funds → investors → leases → financial metrics would enable queries like "show me all properties in Fund III with leases expiring within 12 months where the tenant's credit rating has declined" — queries that require traversing typed relationships, not just vector similarity. This is a 6-12 month initiative but is the architectural foundation for becoming a true operating system.

No permission-aware retrieval. Notion processes **200B+ blocks** with permission-aware AI retrieval — computing block permissions via tree traversal to ensure users only see data they're authorized to access. (ZenML) For REPE, where different team members have access to different funds, deals, and investor information, RAG retrieval must enforce permissions at the chunk level. A user querying "show me our portfolio performance" should only see results from funds they have access to. This requires permission metadata on every chunk and filtering at retrieval time, not just application-level access control. (Microsoft Learn)

No evaluation-driven development loop. Notion employs dedicated AI Data Specialists who run continuous LLM-as-judge evaluation. Databricks provides a Review App where stakeholders interact with deployed agents and provide feedback. (Databricks) Harvey's citation system ensures every answer is grounded to exact sources. (Medium) Winston needs this closed loop: deploy changes → evaluate automatically → compare metrics → ship or revert. Without it, every prompt change, model upgrade, or pipeline modification is a blind deployment.

Missing LP reporting automation. The ILPA Reporting Template v2.0 was released in January 2025, (Gen II) with an implementation deadline of **Q1 2026** for funds commencing operations. (Gen II) This is imminent. Winston's document intelligence and financial modeling capabilities position it to automate the full LP reporting workflow: aggregate property-level financials, calculate fund-level performance metrics (IRR, MOIC, (ACA Group) DPI, RVPI, TVPI per ILPA's new granular and gross-up methodologies), (Gen II) generate capital account statements, and produce ILPA-compliant output. No competitor fully automates this pipeline. This is a **near-term market differentiator** that should be prioritized.

---

8. Performance improvements that compound

Several performance optimizations interact multiplicatively — implementing them together yields substantially more than their individual gains.

Prompt caching is free money. OpenAI automatically caches prompts ≥1,024 tokens with no code changes required, delivering **50% cost reduction and up to 80% latency reduction**. Anthropic offers manual cache configuration with up to **90% cost reduction and 85% latency reduction**. (anthropic) Winston's system prompts + MCP tool definitions almost certainly exceed 1,024 tokens, meaning every request likely qualifies. Structure prompts with static components first (tool definitions, system instructions, few-shot examples) and dynamic components last (conversation history, user query) to maximize cache hit rates. This single change likely saves **30-50% of current API costs** with zero quality impact.

Model tiering across lanes cuts costs 40-60%. The single most impactful cost optimization: assign different models to different lanes based on their requirements.

- **Lane A (Conversational, <1s):** GPT-4o-mini at $0.15/$0.60 per million tokens — 17x cheaper than GPT-4o, sufficient for simple conversation, and fast enough to hit the <1s target
- **Lane B (Tool-backed, <4s):** GPT-4o at $2.50/$10 per million tokens — the current model, appropriate for tool orchestration

- **Lane C (Analytical, <8s):** o4-mini at low reasoning effort — delivers reasoning capability at ~60% the cost of GPT-4o with better analytical performance (Microsoft Community Hub)
- **Lane D (Deep reasoning, <20s):** o4-mini at medium/high reasoning effort or o3 — genuinely superior reasoning for complex financial analysis

RouteLLM research demonstrates that intelligent routing achieves **85%+ cost reduction** on MT-Bench while maintaining quality. (Champaign Magazine) Even a conservative implementation with model tiering saves 40-60%.

Semantic caching for the long tail of repetitive queries. Redis semantic cache with tenant-scoped keys, checking cache before the RAG pipeline, adds only 5-20ms per request but saves 1-5 seconds on cache hits. (Redis) With **61-69% hit rates** achievable for typical enterprise query patterns, (arXiv) this compounds the savings from prompt caching and model tiering. Configure similarity thresholds conservatively (0.95 cosine) and set TTLs per content type: 1 hour for market data, 24 hours for fund documents, 7 days for static reference data.

Dynamic context window management. Implement a TokenBudgetManager that allocates context window capacity dynamically: system prompt (10-15%), tool definitions (10-15%, loaded dynamically), RAG context (30-40%), conversation history (20-30%), and output reservation (10-15%). For conversations exceeding 10 turns, trigger incremental summarization — compress older turns while preserving critical entities and decisions. This prevents context overflow while maintaining coherence in extended analysis sessions.

---

9. Security improvements required for enterprise readiness

Winston's security posture must transform from startup-grade to enterprise-grade before serving institutional REPE firms. The following improvements are ordered by risk severity.

Implement defense-in-depth for prompt injection. This is OWASP's #1 LLM risk. (Vectra AI) With MCP tools capable of database access, document manipulation, and execution actions, a successful injection could cause catastrophic damage. The defense stack requires five layers: **(1)** input validation with regex + ML classifier for injection patterns, **(2)** structured prompts with clear delimiters between system instructions and user content, **(3)** output filtering scanning responses for PII leakage and system prompt echoing, **(4)** tool call validation ensuring arguments are within expected ranges, and **(5)** behavioral monitoring tracking anomalous tool calls and exfiltration attempts. (Sapt) The MCP tool system specifically needs per-tool authorization checks, input validation, output sanitization, and comprehensive audit logging.

Build a PII detection pipeline for financial data. Every request to the LLM should pass through PII detection that identifies SSNs, tax IDs, financial account numbers, investor names associated with deal terms, and property addresses linked to owners. Microsoft Presidio (open-source) or Protecto can mask or tokenize PII before the prompt reaches the model. NER-based detection achieves **93%+ accuracy** on financial documents. (Nature) For REPE data specifically, deal terms, LP commitment amounts, and promote structures are commercially sensitive even when not technically PII — consider field-level encryption for the most sensitive data classes.

Enforce row-level security across all database tables. Supabase Postgres supports RLS natively. Every table containing tenant data — documents, embeddings, conversations, tool results, metrics — needs a tenant_id column with RLS policies enforced at the database level, not just the application layer. This is defense-in-depth: even if application code has a bug that omits tenant filtering, the database blocks cross-tenant access. Nile For the vector store specifically, implement partial HNSW indexes per large tenant to ensure vector search isolation and performance.

Begin SOC 2 preparation now. SOC 2 doesn't have AI-specific criteria, but 2025 auditors expect documentation of AI vendor data handling agreements (confirm OpenAI's no-training guarantee), prompt injection controls, complete AI interaction audit trails (inputs, outputs, model used, tokens consumed, timestamp, tenant, user), change management with prompt version control and approval workflows, and data classification with PII and financial data tagged throughout the pipeline. The AICPA's Trust Service Criteria — Security, Availability, Processing Integrity, Confidentiality, Privacy — all apply to Winston's AI operations.

Implement comprehensive audit logging. Every AI interaction must be logged with sufficient detail for regulatory compliance. This includes the full prompt (with PII masked in logs), the model response, which model was used and which lane, all tool calls with inputs and outputs, RAG retrieval results and relevance scores, token consumption and cost, tenant and user identifiers, and timestamps. These logs serve three purposes: debugging and quality improvement, SEC regulatory compliance, and SOC 2 audit readiness. Retention policy: chat logs for 90 days minimum, audit trails for 7 years (matching SEC recordkeeping requirements for investment advisers).

---

10. Winston v2 architecture blueprint

The target architecture addresses every gap identified in this audit while preserving Winston's existing strengths.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          FRONTEND LAYER                              │
│  Next.js 15 + Vercel AI SDK v5+ + shadcn/ai components               │
│  Structured SSE events │ Token buffering │ Optimistic UI            │
│  Vercel Edge Functions for auth + routing                            │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          AI GATEWAY LAYER                            │
│  Portkey (or LiteLLM) with MCP Gateway                               │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐                │
│  │Multi-    │ │Prompt    │ │Semantic  │ │Cost      │                │
│  │Provider  │ │Caching   │ │Caching   │ │Governance│                │
│  │Fallback  │ │(auto)    │ │(Redis)   │ │& Rate Limit│              │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘                │
│  Token Budget Enforcement │ PII Detection │ Input Guard              │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        INTELLIGENT ROUTER                            │
│  RouteLLM-style classifier + static lane fallback                    │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐                │
│  │Lane A    │ │Lane B    │ │Lane C    │ │Lane D    │                │
│  │GPT-4o-   │ │GPT-4o    │ │o4-mini   │ │o4-mini/o3│                │
│  │mini      │ │          │ │(low)     │ │(med/high)│                │
│  │<1s       │ │<4s       │ │<8s       │ │<20s      │                │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘                │
│  + Lane E: Background/Batch (no latency constraint)                  │
│  + Escalation: A→B→C→D on quality threshold failure                  │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      AGENT ORCHESTRATION LAYER                       │
│  PLAN → CONFIRM → EXECUTE → VERIFY (+ adaptive replan)               │
│  ┌──────────┐ ┌──────────┐ ┌──────────────────┐                     │
│  │Working   │ │Episodic  │ │Procedural Memory │                     │
│  │Memory    │ │Memory (Zep)│ │(cached tool patterns)│               │
│  └──────────┘ └──────────┘ └──────────────────┘                     │
│  LangGraph state machine for Lane D multi-agent flows                │
│  Dynamic MCP tool loading (5-10 per query, not all 12+)              │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│                    ENHANCED RAG PIPELINE                     │
│ Query Rewriting + Multi-Query Generation                     │
│                    ↓                                         │
│ Hybrid Search: BM25 (ParadeDB) + Vector (pgvector)           │
│                    ↓                                         │
│ RRF Fusion (k=60) — over-fetch 50-100 candidates             │
│                    ↓                                         │
│ Metadata Boosting (doc type, recency, fund relevance)        │
│                    ↓                                         │
│ Cross-Encoder Rerank (Cohere Rerank or mxbai-rerank)         │
│                    ↓                                         │
│ Score Threshold → MMR Deduplication → Top-K (5-10)           │
│                                                              │
│ Embeddings: text-embedding-3-large (or Voyage-3-large)       │
│ Contextual Retrieval on all chunks                           │
│ GraphRAG for multi-hop analytical queries (Lane C/D)         │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                    DATA & DOMAIN LAYER                       │
│ ┌──────────────┐ ┌──────────────────┐ ┌──────────────────────┐ │
│ │Supabase      │ │REPE Ontology     │ │Document Intelligence │ │
│ │Postgres      │ │(Properties,      │ │(Loan, Lease, PSA, JV,│ │
│ │+ pgvector    │ │Funds, LPs,       │ │Appraisal, IC Memo,   │ │
│ │+ ParadeDB    │ │Leases, Deals)    │ │Operating Agreement)  │ │
│ │+ RLS         │ │                  │ │                      │ │
│ └──────────────┘ └──────────────────┘ └──────────────────────┘ │
│ ┌──────────────────────┐ ┌──────────────────────────────────┐ │
│ │Financial Engine      │ │LP Reporting Engine               │ │
│ │Waterfall/Promote     │ │ILPA v2.0 Template Automation     │ │
│ │IRR/MOIC/DCF          │ │ESG (GRESB/SFDR) Integration      │ │
│ │Debt Sizing           │ │Quarterly Report Generation       │ │
│ └──────────────────────┘ └──────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│              OBSERVABILITY & EVALUATION LAYER                │
│ Langfuse (self-hosted): Tracing + Prompt Management          │
│ DeepEval: CI/CD regression testing + red teaming             │
│ RAGAS: Continuous RAG quality monitoring                     │
│ Helicone: Cost tracking + budget alerts + caching            │
│ Per-tenant dashboards: cost, usage, quality metrics          │
└──────────────────────────────────────────────────────────────┘
```

The v2 architecture preserves every existing strength — the lane-based routing, PLAN→CONFIRM→EXECUTE loop, MCP tool system, and domain-specific document intelligence — while adding the infrastructure layer that transforms Winston from a capable prototype into a production enterprise platform. The key insight from benchmarking is that **the best AI platforms are distinguished not by their core AI capabilities but by their governance, observability, and reliability infrastructure**.

---

11. Strategic roadmap: from current state to category leader

Phase 1: Foundation hardening (days 1–90)

Week 1-2: Eliminate existential risks. Deploy Langfuse (self-hosted on Railway) for tracing and prompt management. Implement OpenAI prompt caching by restructuring prompts with static prefixes — this costs nothing and immediately saves 30-50% on API costs. Add Anthropic Claude as a secondary provider via LiteLLM, with automatic fallback when OpenAI is unavailable.

Week 3-4: Fix the RAG pipeline. Correct the pipeline ordering (hybrid search first, score threshold after reranking). Increase over-fetch from 5 to 50 candidates. Add query rewriting as a pre-retrieval step. These are code changes measured in days, not weeks, with immediate quality improvement.

Week 5-8: Security fundamentals. Implement row-level security on all database tables with `tenant_id` enforcement. Add input validation for prompt injection patterns. Deploy PII detection on the prompt pipeline. Begin documenting AI controls for SOC 2 readiness. Set up comprehensive audit logging.

Week 9-12: Model tiering and cost governance. Deploy GPT-4o-mini for Lane A, keep GPT-4o for Lane B, add o4-mini for Lanes C/D. Implement per-tenant cost tracking with Helicone virtual keys or Langfuse cost attribution. Deploy Redis semantic cache with tenant-scoped keys. Set up budget alerts at 50/80/100% of tenant allocation. **Expected combined cost reduction: 50-70%.**

Phase 1 delivers: Multi-provider resilience, correct RAG pipeline, basic security posture, cost optimization, and production observability. These changes position Winston for enterprise pilot deployments.

Phase 2: Enterprise platform maturity (months 4–12)

Months 4-6: Evaluation-driven development. Build a domain-specific evaluation dataset of 200+ QA pairs from actual REPE documents. Integrate DeepEval into CI/CD for automated regression testing. Deploy RAGAS for continuous production monitoring of faithfulness, context precision, and answer relevancy. Implement monthly red team testing for prompt injection and cross-tenant data leakage.

Months 4-6: Enhanced RAG. Upgrade embedding model to text-embedding-3-large or Voyage-3-large (requires full re-embedding). Add Anthropic's contextual retrieval to all chunks during re-embedding. Deploy ParadeDB for BM25 to enable true hybrid search. Implement LongLLMLingua for context compression on retrieved chunks.

Months 6-9: Agent intelligence. Add VERIFY step to agent loop. Implement adaptive replanning with bounded retry. Deploy Zep for cross-session memory. Build dynamic MCP tool loading with semantic tool selection. Implement LangGraph for Lane D complex multi-step workflows with checkpointing and durable execution.

Months 9-12: Domain differentiation. Launch ILPA v2.0 automated LP reporting. Build the REPE ontology layer mapping properties, funds, investors, leases, and deals as typed entities with relationships. Implement permission-aware retrieval that respects fund-level and deal-level access controls. Deploy GraphRAG for multi-hop analytical queries across the ontology.

Phase 2 delivers: A production-hardened enterprise platform with continuous quality monitoring, sophisticated RAG, intelligent agents with memory, and domain-specific features (LP reporting, ontology-driven queries) that no competitor offers.

Phase 3: Category definition (years 2–3)

The AI-native ARGUS. Build the financial modeling engine that ingests documents, extracts terms, populates DCF models, runs waterfall calculations, and generates institutional-grade outputs — all through natural language. ARGUS processes ~$10T+ in annual transaction volume but has zero AI capability. Winston's document intelligence + financial engine + AI interface could capture the next generation of this market. The key insight: don't replace ARGUS directly — offer ARGUS-compatible outputs while providing AI-native capabilities ARGUS cannot match (automated data entry, natural language querying, instant sensitivity analysis, document-to-model automation).

Multi-agent deal teams. For complex workflows — "analyze this OM, build an underwriting model, run sensitivities, compare to our existing portfolio exposure, and generate an IC memo draft" — deploy specialized agent teams coordinated by a supervisor agent. One agent handles document extraction, another builds the financial model, another performs market analysis, and a synthesis agent produces the IC memo. This mirrors how actual REPE deal teams operate and represents the true "operating system" vision.

Predictive intelligence layer. Move beyond reactive AI (answer questions about existing data) to proactive AI (surface insights before users ask). Monitor lease expirations, covenant compliance, market shifts, and portfolio concentration. Alert investment teams when conditions change. This requires the ontology layer from Phase 2, continuous data integration, and predictive models trained on REPE-specific patterns.

Infrastructure evolution. Migrate from Railway to AWS/GCP for full SOC 2 compliance posture, VPC isolation, KMS encryption, and CloudTrail audit logging. Evaluate dedicated vector databases if corpus exceeds 10M vectors. Consider self-hosted models (fine-tuned on REPE documents) for the most sensitive workflows where data cannot leave the infrastructure.

Phase 3 delivers: A category-defining platform that institutional REPE firms cannot operate without. The **$34 billion efficiency opportunity** that Morgan Stanley estimates for AI in real estate is concentrated in exactly the workflows Winston targets. The firm that builds the integrated AI operating system for REPE — from document intake through financial modeling to investor reporting — wins a generational market shift.

Conclusion

Winston's architecture reflects the decisions of a technically thoughtful team that understands both the AI platform landscape and the REPE domain. The 4-lane routing model, human-in-the-loop confirmation pattern, comprehensive MCP tool system, and domain-specific document intelligence are genuine strengths that most competitors lack. The platform's weaknesses are almost entirely in the **infrastructure and governance layer** — observability, security, cost management, evaluation — rather than in core AI capabilities. This is the better problem to have: infrastructure can be added incrementally without rearchitecting the core.

The competitive window is real but narrowing. **88% of real estate investors are piloting AI**, but **60%+ remain unprepared to scale beyond pilots**. No competitor offers the integrated operating system vision Winston targets. The firms that will dominate this market are the ones that combine deep domain intelligence with production-grade enterprise infrastructure. Winston has the domain intelligence. This roadmap provides the path to the infrastructure.

The single most important insight from benchmarking 10 leading AI platforms: **the best systems are not the ones with the most sophisticated AI — they are the ones with the most rigorous evaluation, observability, and governance infrastructure.** Palantir, Databricks, Harvey, and Notion all invest more engineering effort in measuring and governing their AI than in the AI itself. Winston's v2 architecture should follow this pattern: build the measurement infrastructure first, then optimize what you can measure.